

Operating Systems — Midterm Practice Exam — ANSWER KEY

Q1 Interrupt, I/O Operation (25 pts)

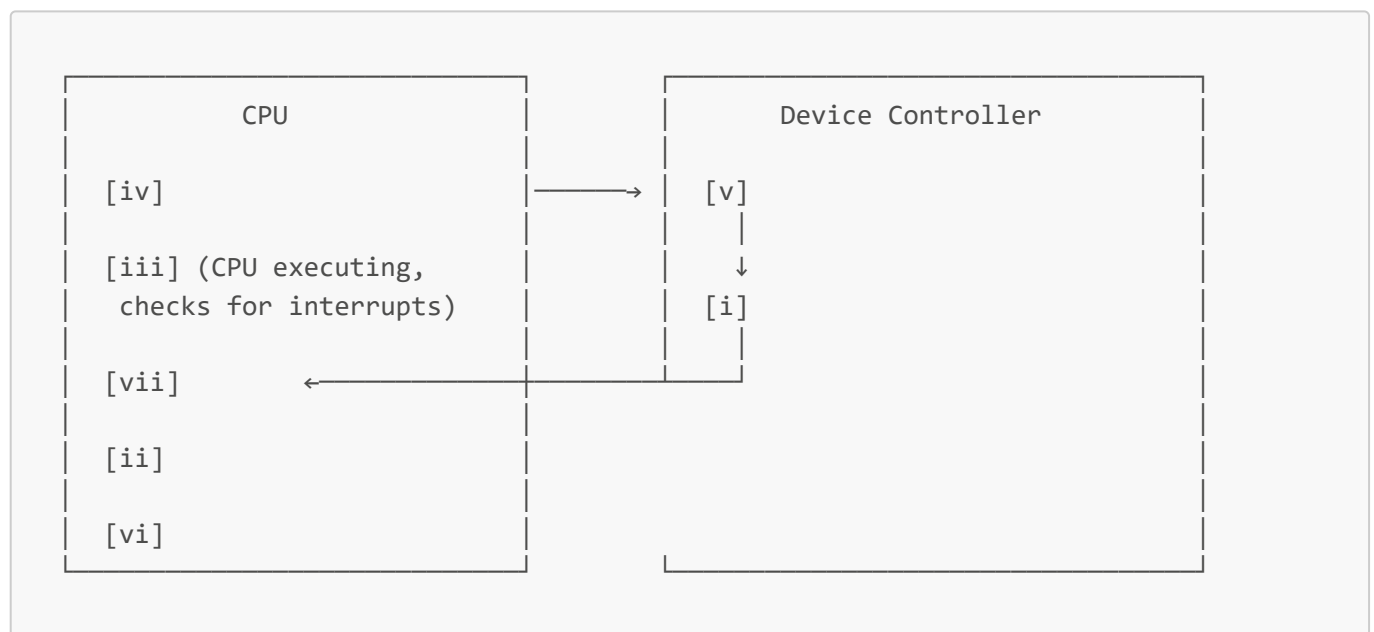
(a) (5 pts) Advantage of interrupt-based operations over busy-waiting

Answer:

With **busy-waiting (polling)**, the CPU continuously loops checking whether the I/O device is ready. This wastes a large amount of CPU time because the CPU is stuck in a loop doing no useful work.

With **interrupt-based** I/O, the CPU can continue executing other processes or instructions while the I/O device performs its operation. When the device completes, it sends an **interrupt signal** to notify the CPU. This significantly increases **CPU utilization** since the CPU is not idle during I/O operations.

(b) (10 pts) Interrupt-driven I/O flowchart



Explanation:

- **(iv)** Device driver initiates I/O, sends command to device controller
 - **(iii)** CPU continues executing, checks interrupt-request line after each instruction
 - **(v)** Controller performs I/O operation (reads/writes device data)
 - **(i)** I/O complete → generates interrupt signal
 - **(vii)** CPU receives interrupt, transfers control to interrupt handler
 - **(ii)** Interrupt handler processes data, returns from interrupt
 - **(vi)** CPU resumes execution of interrupted process
-

(c) (5 pts) Device controller vs device driver

Device controller: A **hardware** component that sits between the device and the CPU. It controls the actual device operation and has a **local buffer** to temporarily store data being transferred to/from the device.

Device driver: A **software** module (part of the OS kernel) that tells the OS how to communicate with a specific device controller. It provides a **uniform interface** so the OS can interact with different devices in a standardized way.

(d) (5 pts) Benefit of using interrupt vector

Answer:

The interrupt vector (table) stores the **addresses of interrupt-specific handlers** for each type of interrupt. When an interrupt occurs, the CPU uses the interrupt number as an index to directly look up the corresponding handler address in the table. This eliminates the need to **sequentially search through all handlers**, significantly speeding up interrupt processing.

Q2 System Calls, Operating System Design (20 pts)

(a) (10 pts) How a system call is invoked using an API function

Answer:

1. The user program calls an API function (e.g., `read()`).
 2. The API function triggers a **software interrupt (trap)**, which causes the CPU to switch from **user mode** to **kernel mode (dual-mode operation)**.
 3. The **system call number (index)** is placed into a designated register (e.g., EAX).
 4. The CPU uses this index to look up the **table of code pointer** (system call table) to find the memory address of the corresponding kernel function.
 5. The CPU jumps to that address and executes the kernel service.
 6. After the service completes, the CPU switches back to **user mode** and returns the result to the calling program.
-

(b) (5 pts) Why programmers prefer API over direct system calls

Answer:

1. **Portability:** API functions provide a consistent interface across different operating systems. A program written using POSIX API can run on any POSIX-compliant OS without modification.
 2. **Simplicity:** API functions are easier to use than raw system calls. Programmers do not need to understand the low-level details of how each system call works, such as register conventions or system call numbers.
-

(c) (5 pts) Pros and cons of layered OS structure

Pros: Modular design makes it **easy to debug and verify**. Each layer can be verified independently from the lowest layer (Layer 0, hardware) upward. If a bug exists in a layer, you know the layers below are already correct, so the problem must be in that layer.

Cons:

1. It is **difficult to correctly define** what belongs in each layer, since many components have interdependencies.
2. **Poor performance** due to overhead — each service request must traverse multiple layers, with each layer adding function call overhead.

Q3 Process State (20 pts)

(a) (10 pts) Process state diagram labels

```

admitted → ready      : (none – admitted by long-term scheduler)
ready → running       : (ii) be dispatched by a short-term scheduler
running → ready       : (i) be interrupted
running → waiting     : (iii) perform I/O
waiting → ready       : (iv) have I/O completed
running → terminated  : (v) exit
  
```

(b) (10 pts) Process state in each situation

#	Situation	State
i	after it is created (forked) by a parent process	ready
ii	before it is selected by a short-term scheduler	ready
iii	after it is selected by a short-term scheduler	running
iv	before it invokes a blocking system call	running
v	after it invokes a blocking system call	waiting
vi	after the blocking system call is complete	ready
vii	after it is interrupted by a hardware interrupt	ready
viii	after its time slice expired	ready

Common mistake: vii and viii are **ready**, NOT waiting. The process is not waiting for any I/O event — it was simply preempted and can run again immediately.

Q4 Multithreaded Programming (20 pts)

(a) (5 pts) Why process creation is costly compared to thread creation

Answer:

When creating a new process, the OS must **duplicate the entire address space** of the parent process, including the text (code), data, and heap sections. This involves significant memory allocation and copying.

In contrast, creating a thread only requires allocating a new **thread ID, program counter, register set, and stack**. Threads within the same process **share** the code, data, heap, and open files. This makes thread creation much cheaper — in Solaris, thread creation is approximately **30 times faster** than process creation.

(b) (5 pts) Many-to-One vs One-to-One thread models

Many-to-One:

- Multiple user threads map to **one** kernel thread.
- **Pros:** Thread management is done entirely in user space, so it is efficient and requires no system calls for thread operations.
- **Cons:** If any one thread makes a **blocking system call**, the entire process (all threads) is blocked. Also, threads **cannot run in parallel** on multiprocessors since there is only one kernel thread.

One-to-One:

- Each user thread maps to **one** kernel thread.
 - **Pros:** One thread blocking does **not affect** others. Threads can **truly run in parallel** on multiprocessors.
 - **Cons:** Creating a user thread requires creating a corresponding kernel thread, resulting in **higher overhead**. The number of threads may be limited. Used by **Linux and Windows**.
-

(c) (5 pts) Problem that Scheduler Activations solves

Answer:

In the **Many-to-Many** thread model, when a user thread makes a **blocking system call**, the kernel blocks the underlying kernel thread. Since multiple user threads share kernel threads, the kernel has no way to notify the thread library about this blocking. As a result, the thread library cannot schedule another user thread on the blocked LWP, causing **unnecessary CPU idle time** — other ready user threads cannot execute even though the CPU is available.

(d) (5 pts) How Scheduler Activations work

Answer:

1. A user thread is about to make a **blocking system call**.
 2. The kernel detects this and, instead of silently blocking, sends an **upcall** (upward notification) to the **thread library**.
 3. The **upcall handler** in the thread library receives the notification, **saves the state** of the blocked thread (PC, registers, etc.), and marks it as blocked.
 4. The thread library's **scheduler** then selects another **ready user thread** from its ready queue and schedules it on an available LWP to execute.
 5. When the blocking I/O completes, the kernel sends another **upcall** to the thread library.
 6. The upcall handler marks the original thread as **ready** and places it back in the ready queue for future scheduling.
-

Q5 Code Fill-in (15 pts)

Part A — Shared Memory (8 pts)

Blank	Answer	Explanation
(A)	<code>shmget</code>	Create/get a shared memory segment
(B)	<code>shmat</code>	Attach the shared memory to the process address space
(C)	<code>0</code>	Initialize shared memory to 0
(D)	<code>fork</code>	Create a child process
(E)	<code>2 * x</code>	First child computes the 2x term
(F)	<code>3 * y</code>	Second child computes the 3y term
(G)	<code>wait</code>	Parent waits for each child to finish
(H)	<code>shmctl</code>	Remove the shared memory segment

Part B — Shell Interface (7 pts)

Blank	Answer	Explanation
(A)	<code>true</code>	Command ends with '&', so it should run in background
(B)	<code>false</code>	No '&', so it runs in foreground
(C)	<code>fork</code>	Create a child process to execute the command
(D)	<code>execlp</code>	Replace child process image with the command (also accept <code>execvp</code> , <code>exec</code>)
(E)	<code>false</code>	If NOT background, parent should wait
(F)	<code>wait</code>	Parent waits for the foreground child to finish

Bonus CPU Scheduling (+10 pts)

(a) (6 pts) Gantt Chart — Multilevel Feedback Queue

Setup reminder:

- Q2 (highest): Round-Robin, quantum = 4 ms
- Q1 (middle): Round-Robin, quantum = 6 ms
- Q0 (lowest): FCFS
- A process returning from I/O re-enters Q2.
- A high-level process preempts a low-level process.
- A process preempted by timeout is demoted to the next lower level.

Step-by-step execution:

Time	Event
------	-------

Time	Event
0–2	P ₁ runs (Q2), uses 2 ms of its 8 ms CPU burst
2	P ₂ arrives → enters Q2. P ₁ and P ₂ are both in Q2; P ₁ continues (already running, no preemption within same queue)
2–4	P ₁ runs (Q2), total 4 ms used → Q2 quantum expired , P ₁ demoted to Q1 (4 ms burst remaining)
4–6	P ₂ runs (Q2), uses 2 ms → P ₂ CPU burst done, P ₂ goes to I/O (2 ms, finishes at t=8)
6–8	P ₁ runs (Q1), uses 2 ms (4 ms remaining → 2 ms remaining)
8	P ₂ returns from I/O → enters Q2 (higher priority). P ₁ is preempted (stays in Q1, 2 ms remaining). P ₁ is NOT demoted further since it was preempted by a higher-priority process, not by timeout.
8–12	P ₂ runs (Q2), uses 4 ms → Q2 quantum expired , P ₂ demoted to Q1 (12 ms remaining)
12–14	P ₁ runs (Q1), uses 2 ms → P ₁ first CPU burst done (total 8 ms). P ₁ goes to I/O (4 ms, finishes at t=18)
14–20	P ₂ runs (Q1), uses 6 ms → Q1 quantum expired , P ₂ demoted to Q0 (6 ms remaining)
18	P ₁ returns from I/O → enters Q2 (during P ₂ 's Q1 run, but P ₂ finishes Q1 at t=20)
18–20	Wait — P ₁ returned at t=18 and enters Q2, which is higher priority. P ₂ is preempted at t=18. P ₁ runs from Q2.

Let me redo this more carefully:

Time	Running	Queue	Detail
0–2	P ₁	Q2	P ₁ starts, 2/4 ms of Q2 quantum used
2–4	P ₁	Q2	P ₂ arrives at t=2 (enters Q2). P ₁ continues to finish its Q2 quantum. At t=4: P ₁ timeout → demoted to Q1 (4 ms of first burst remaining)
4–6	P ₂	Q2	P ₂ runs in Q2. At t=6: P ₂ finishes first CPU burst (2 ms). P ₂ → I/O (2 ms, done at t=8)
6–8	P ₁	Q1	P ₁ runs from Q1 (only process). Uses 2 ms (2 ms remaining of first burst)
8	—	—	P ₂ returns from I/O → enters Q2 (preempts P ₁ in Q1)
8–12	P ₂	Q2	P ₂ runs. At t=12: Q2 quantum (4 ms) expires → P₂ demoted to Q1 (12 ms remaining)
12–14	P ₁	Q1	P ₁ resumes in Q1. Uses 2 ms → first CPU burst complete. P ₁ → I/O (4 ms, done at t=18)
14–18	P ₂	Q1	P ₂ runs from Q1. Uses 4 ms (8 ms remaining)
18	—	—	P ₁ returns from I/O → enters Q2 (preempts P ₂ in Q1). P ₂ used 4/6 of Q1 quantum.

Time	Running	Queue	Detail
18–22	P ₁	Q2	P ₁ runs from Q2. At t=22: Q2 quantum (4 ms) expires → P₁ demoted to Q1 (4 ms remaining)
22–24	P ₂	Q1	P ₂ resumes in Q1 with 2 ms remaining of its Q1 quantum. At t=24: Q1 quantum expires → P₂ demoted to Q0 (6 ms remaining)
24–28	P ₁	Q1	P ₁ runs from Q1. Uses 4 ms → second CPU burst complete. P₁ done at t=28.
28–34	P ₂	Q0	P ₂ runs from Q0 (FCFS). Uses 6 ms → P₂ done at t=34.

Gantt Chart:

Time: 0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34

| | | | | | | | | | | | | | | | | |

| | | | | | | | | | | | | | | | | |

[P₁ Q2][P₂ Q2][P₁ Q1][P₂ Q2][P₁Q1][P₂ Q1][P₁ Q2][P₂] [P₁ Q1][P₂ Q0]

|---P₁---|---P₂---|---P₁---|-----P₂-----|---P₁---|---P₂---|---P₁-----

|P₂-Q1|---P₁-----|-----P₂-----|

t=0 t=4 t=6 t=8 t=12 t=14 t=18 t=22 t=24

t=28 t=34

Simplified:

0- 4: P₁ (Q2)

4- 6: P₂ (Q2)

6- 8: P₁ (Q1)

8-12: P₂ (Q2)

12-14: P₁ (Q1)

14-18: P₂ (Q1)

18-22: P₁ (Q2)

22-24: P₂ (Q1)

24-28: P₁ (Q1)

28-34: P₂ (Q0)

Note: P₁ I/O occurs at t=14–18. P₂ I/O occurs at t=6–8.

(b) (4 pts) Average waiting time

Process	Arrival	Finish	Total CPU Burst	Turnaround Time	Waiting Time
P ₁	0	28	8 + 8 = 16 ms	28 – 0 = 28 ms	28 – 0 – 16 – 4 (I/O) = 8 ms
P ₂	2	34	2 + 16 = 18 ms	34 – 2 = 32 ms	34 – 2 – 18 – 2 (I/O) = 12 ms

Waiting Time = Finish – Arrival – Total CPU Burst – I/O Burst

Average Waiting Time = $(8 + 12) / 2 = 10$ ms

End of answer key.